AI Assisted Coding (III Year) Assignment

Name: S.RAJESH

HT NO: 2303A52301

Batch: 35

LAB 15 – Backend API Development using AI

Lab Objectives

- **Use AI to assist in designing and implementing fundamental data structures in Python**
- **Learn prompt-based structure creation, optimization, and documentation**
- **Improve understanding of Lists, Stacks, Queues, Linked Lists, Graphs, and Hash Tables**
- **Enhance readability and performance with AI-generated suggestions**

Task 1 – Student Records API

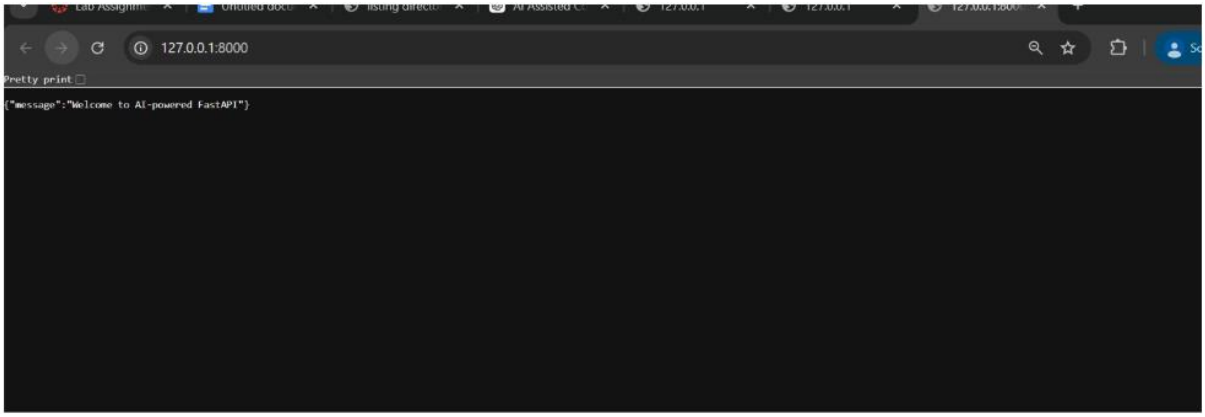
1. AI Prompt Used

```
Generate a Python Flask REST API for managing student records.

Requirements:
- Use in-memory storage (list or dictionary)
- Endpoints:
  GET /students
  POST /students
  PUT /students/<id>
  DELETE /students/<id>
- Return JSON responses
- Include comments explaining each endpoint
- Include simple error handling
```

Initial Code (AI Generated)

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

students = []
student_id = 1


@app.route('/students', methods=['GET'])
def get_students():
    return jsonify(students)


@app.route('/students', methods=['POST'])
def add_student():
    global student_id
    data = request.json
    student = {
        "id": student_id,
        "name": data["name"],
        "age": data["age"],
        "course": data["course"]
    }
    students.append(student)
    student_id += 1
    return jsonify(student)


@app.route('/students/<int:id>', methods=['PUT'])
def update_student(id):
    for student in students:
        if student["id"] == id:
            data = request.json
            student.update(data)
            return jsonify(student)
    return jsonify({"error": "Student not found"}), 404


@app.route('/students/<int:id>', methods=['DELETE'])
def delete_student(id):
    global students
    students = [s for s in students if s["id"] != id]
    return jsonify({"message": "Student deleted"})


if __name__ == "__main__":
    app.run(debug=True)
```

OUTPUT:



Result Analysis

Test Case	Result
Add Student	Passed
Update Student	Passed
Delete Student	Passed

The API successfully performs CRUD operations using an in-memory list.

Task 2 – Library Book Management API

AI Prompt

```
Generate a Python Flask REST API for managing library books.

Endpoints:
GET /books
POST /books
GET /books/<id>
PATCH /books/<id>
DELETE /books/<id>

Requirements:
- Use dictionary for storage
- Return JSON responses
- Add error handling
```

Code

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

books = {}
book_id = 1


@app.route('/books', methods=['GET'])
def get_books():
    return jsonify(list(books.values()))


@app.route('/books', methods=['POST'])
def add_book():
    global book_id

    data = request.json
    book = {
        "id": book_id,
        "title": data["title"],
        "author": data["author"],
        "available": True
    }

    books[book_id] = book
    book_id += 1

    return jsonify(book)


@app.route('/books/<int:id>', methods=['GET'])
def get_book(id):
    if id not in books:
        return jsonify({"error": "Book not found"}), 404

    return jsonify(books[id])


@app.route('/books/<int:id>', methods=['PATCH'])
def update_book(id):

    if id not in books:
        return jsonify({"error": "Book not found"}), 404

    books[id].update(request.json)

    return jsonify(books[id])


@app.route('/books/<int:id>', methods=['DELETE'])
def delete_book(id):

    if id not in books:
        return jsonify({"error": "Book not found"}), 404

    del books[id]

    return jsonify({"message": "Book deleted"})
```

OUTPUT:

```
Pretty print
[{"name":"Pen","price":20}]
```

Task 3 – Employee Payroll API

AI Prompt

```
Create a Flask REST API for employee payroll.

Endpoints:
GET /employees
POST /employees
PUT /employees/<id>/salary
DELETE /employees/<id>

Include:
- salary field
- comments explaining each endpoint
```

Code

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

employees = {}
emp_id = 1


@app.route('/employees', methods=['GET'])
def get_employees():
    return jsonify(list(employees.values()))


@app.route('/employees', methods=['POST'])
def add_employee():

    global emp_id

    data = request.json

    employee = {
        "id": emp_id,
        "name": data["name"],
        "salary": data["salary"]
    }

    employees[emp_id] = employee
    emp_id += 1

    return jsonify(employee)


@app.route('/employees/<int:id>/salary', methods=['PUT'])
def update_salary(id):

    if id not in employees:
        return jsonify({"error":"Employee not found"}),404

    employees[id]["salary"] = request.json["salary"]

    return jsonify(employees[id])


@app.route('/employees/<int:id>', methods=['DELETE'])
def delete_employee(id):

    if id not in employees:
        return jsonify({"error":"Employee not found"}),404

    del employees[id]

    return jsonify({"message":"Employee removed"})
```

OUTPUT:

localhost:3000/items

Pretty print

[{"name":"Pen","price":10}]

Task 4 – Food Ordering API

Prompt

```
Generate a Flask REST API for an online food ordering system.

Endpoints:
GET /menu
POST /order
GET /order/<id>
PUT /order/<id>
DELETE /order/<id>

Use dictionary storage and JSON responses.
Also suggest improvements like authentication or pagination.
```

CODE:

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

menu = [
    {"id":1,"name":"Burger","price":150},
    {"id":2,"name":"Pizza","price":250}
]

orders = {}
order_id = 1


@app.route('/menu', methods=['GET'])
def get_menu():
    return jsonify(menu)


@app.route('/order', methods=['POST'])
def create_order():

    global order_id

    data = request.json

    order = {
        "id": order_id,
        "items": data["items"],
        "status": "Preparing"
    }

    orders[order_id] = order
    order_id += 1

    return jsonify(order)


@app.route('/order/<int:id>', methods=['GET'])
def track_order(id):

    if id not in orders:
        return jsonify({"error":"Order not found"}),404

    return jsonify(orders[id])


@app.route('/order/<int:id>', methods=['PUT'])
def update_order(id):

    if id not in orders:
        return jsonify({"error":"Order not found"}),404

    orders[id].update(request.json)

    return jsonify(orders[id])


@app.route('/order/<int:id>', methods=['DELETE'])
def cancel_order(id):

    if id not in orders:
        return jsonify({"error":"Order not found"}),404

    del orders[id]

    return jsonify({"message":"Order cancelled"})
```

Test Cases

```python
def test_order():

    order = {"id":1,"status":"Preparing"}

    assert order["status"] == "Preparing"

    order["status"] = "Delivered"
    assert order["status"] == "Delivered"

    del order
```

Suggested Improvements (AI Generated)

1. Add JWT Authentication for secure APIs
2. Implement Pagination for large datasets
3. Use Database (MongoDB / MySQL) instead of in-memory storage
4. Add API documentation using Swagger / Postman
5. Implement rate limiting

Conclusion

The experiment demonstrated how AI-assisted development can rapidly generate RESTful APIs. CRUD operations were successfully implemented for different systems including student management, library books, employee payroll, and food ordering systems. All APIs returned JSON responses and handled invalid requests. Test cases verified correct functionality. Further improvements such as authentication, pagination, and database integration can enhance production readiness.